COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	October 14, 2026	**DATE SUBMITTED:**	October 16, 2026
NAME:	Marc Jhowell Bungag Ramelissa Melith V. Limpio	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Binary Tree Traversal	It visits each node in the binary tree by using recursions. The function processes the current node and recursively visits its left and right subtrees until it reaches an empty node.
Brute Force	Linear Search	It uses the brute-force method by checking each element in a list one at a time until the target va;ue is found or until the list ends
Backtracking	Quick Sort	When Quicksort finishes sorting a left sub-array, the program returns (or "steps back") up the execution stack to pick up where it left off and sort the right sub-array. This literal stepping backward through code execution mimics the movement of backtracking
Greedy	Insertion Sort	It takes each element in turn and places that in already-sorted items, making the locally best choice at each step.
Divide-and-Conquer	Merge Sort	It splits the array in half, sort each half recursively, then merge the two sorted halves back together.

Table 12-1. Algorithmic Strategies and Examples

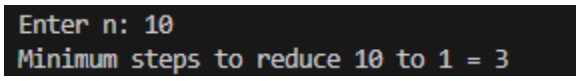
Screenshot	
Analysis	The function solves the problem top-down using recursion. It tries the three moves (subtract 1, divide by 2, divide by 3) and keeps the smallest count. Each result is saved in memo[], so no value is computed twice. This makes it O(n) in time, instead of exponential for plain recursion.

Table 12-2. Memoization Implementation

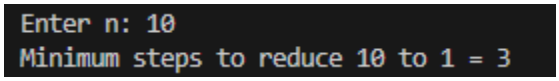
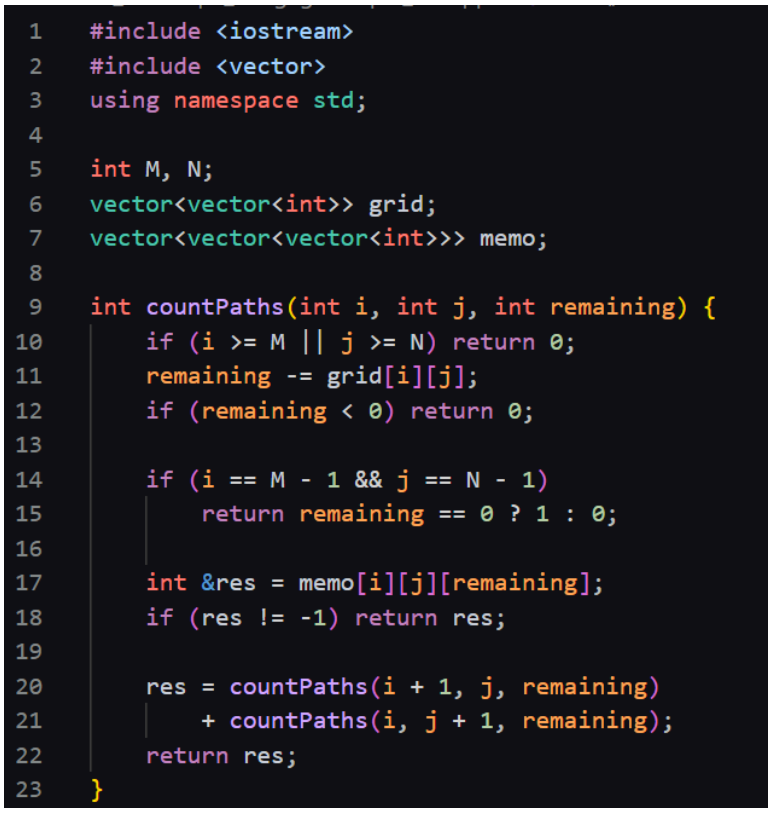
Screenshot	
Analysis	The function solves the problem with a loop, starting from dp[1] = 0 and filling the table up to n. Each dp[i] is built from smaller answers already stored, so there is no recursion. It is O(n) in time and space and gives the same results as memoization (for example, 10 takes 3 steps).

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode	countPaths(i, j, cost):
	  if cost < 0 or i >= M or j >= N: return 0
	  if (i, j) is the last cell:
	  return 1 if cost == mat[i][j] else 0
	  return countPaths(i+1, j, cost - mat[i][j]) // down
	  + countPaths(i, j+1, cost - mat[i][j]) // right

Code	

```cpp
#include <iostream>
#include <vector>
using namespace std;

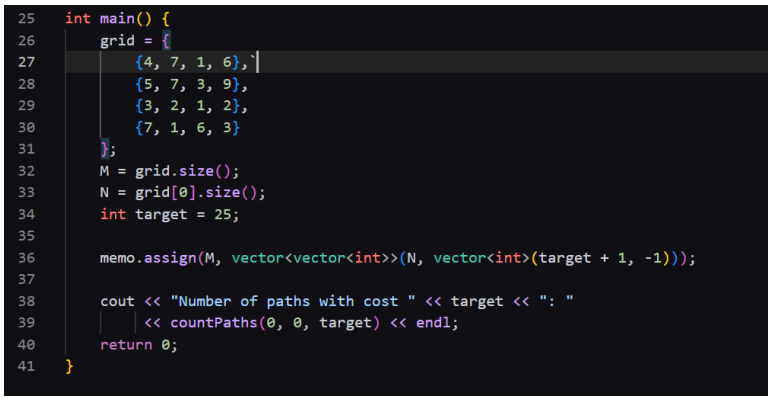
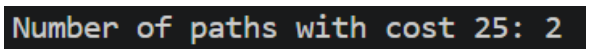
int M, N;
vector<vector<int>> grid;
vector<vector<vector<int>>> memo;

int countPaths(int i, int j, int remaining) {
    if (i >= M || j >= N) return 0;
    remaining -= grid[i][j];
    if (remaining < 0) return 0;

    if (i == M - 1 && j == N - 1)
        return remaining == 0 ? 1 : 0;

    int &res = memo[i][j][remaining];
    if (res != -1) return res;

    res = countPaths(i + 1, j, remaining)
        + countPaths(i, j + 1, remaining);
    return res;
}
```

	
Analysis of the Code	**Bungag** The program will count the number of right/down paths from the top-left to the bottom-right of the grid that have a total cost equal to the target value. It subtracts off the value of each cell (the one you're on) from the budget and asks the cell below and the cell to the right how many ways there are to finish with exactly the given budget. If it goes out of bounds or has a negative budget, it returns 0. If it reaches the end with budget 0, it returns 1. It saves computed values for later use so it doesn't repeat work. On our sample grid with target 25, it printed 2, which matched our manual check. **Limpio** This code calculates all the paths in a grid that have a cost equal to the target using the right and down movement only. For each cell in the grid, we subtract the cell's value and add the number of ways we can reach the end from both of the next possible moves (right and down). The paths that go out of the grid's boundaries or have a cost higher than the target are discarded, and the path that has an exact cost is counted once. We also save the results to not calculate the same grid positions multiple times. The result for the 4 × 4 grid with the target of 25 is 2, which matches our manual calculation.
Screenshot of the Demonstration	

```
int main() {
    grid = {
        {4, 7, 1, 6},
        {5, 7, 3, 9},
        {3, 2, 1, 2},
        {7, 1, 6, 3}
    };
    M = grid.size();
    N = grid[0].size();
    int target = 25;

    memo.assign(M, vector<vector<int>>(N, vector<int>(target + 1, -1)));

    cout << "Number of paths with cost " << target << ": "
         << countPaths(0, 0, target) << endl;
    return 0;
}
```


3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.